LLMs FOR SOCIAL SCIENCE

From foundations to frontier methods

Course Overview

This workshop provides social scientists with a deep, practical understanding of large language models, from the fundamentals of how they encode and process language, through to cutting-edge research applications. Each day combines conceptual depth with hands-on exercises, building skills progressively.

Prerequisites

Participants should be able to read and write Python code at a beginner to intermediate level (desirable). A Google account is required for Google Colab, which we will use for hands-on exercises.

Schedule

Module 1: Foundations – From Embeddings to Transformers

Module 2: From Models to Tools – Post-Training · Prompting & Reasoning · Evaluating Models

Module 3: Deploying for Research – Fine-Tuning & Deployment · Text Classification & Validation

Module 4: Social Science Applications – Information Extraction & RAG

Module 5: Agentic Workflows – Agents & Autonomous Workflows · Capstone Project

MODULE 1: FOUNDATIONS

Understanding the architecture, training, and mechanics that make modern language models possible.

From Embeddings to Transformers

Core Concepts

Word embeddings and distributional semantics: how meaning emerges from context, vector spaces and the geometry of meaning. Tokenization (BPE, WordPiece) and how models actually see text; why tokenization choices determine model capabilities. Language modeling as next-token prediction: the surprisingly powerful core paradigm. The Transformer: self-attention as the breakthrough, multi-head attention, and positional encodings. Scaling laws: the predictable relationship between compute, data, parameters, and performance. Practical implications: context windows, the KV cache, and why API costs scale the way they do.

Core Readings

- Mikolov, T., Chen, K., Corrado, G. & Dean, J. (2013). *Efficient Estimation of Word Representations in Vector Space.* The foundational Word2Vec paper.
- Vaswani, A. et al. (2017). *Attention Is All You Need.* NeurIPS. The Transformer architecture.
- Radford, A. et al. (2019). *Language Models are Unsupervised Multitask Learners.* OpenAI. GPT-2 and zero-shot emergence.
- Kaplan, J. et al. (2020). *Scaling Laws for Neural Language Models.* Power-law relationships between scale and performance.

Further Resources

› Jay Alammar: "The Illustrated Transformer"; the canonical visual walkthrough

› 3Blue1Brown: "Attention in Transformers, visually explained"; intuitive geometric explanation

› Brendan Bycroft's LLM Visualization; interactive 3D transformer walkthrough

› Hugging Face Tokenizer Playground; compare how different tokenizers split text

Social Science Applications

- Caliskan, A., Bryson, J. J. & Narayanan, A. (2017). *Semantics derived automatically from language corpora contain human-like biases.* Science. Word embeddings reflect cultural biases.
- Kozlowski, A. C., Taddy, M. & Evans, J. A. (2019). *The Geometry of Culture.* ASR. Vector space geometry for mapping cultural dimensions.

MODULE 2: FROM MODELS TO TOOLS

How raw language models become useful assistants, and the skills needed to work with them effectively.

Post-Training: Making Models Useful

Core Concepts

The base model → assistant pipeline: why raw pre-trained models are "weird" and how post-training fixes this. Supervised Fine-Tuning (SFT) on instruction-response pairs. Reinforcement Learning from Human Feedback (RLHF): reward modeling and PPO. Direct Preference Optimization (DPO) as a simpler alternative. Constitutional AI and safety training. What post-training changes and what it doesn't (capabilities vs. tendencies).

Core Readings

- Ouyang, L. et al. (2022). *Training language models to follow instructions with human feedback.* NeurIPS. The InstructGPT/RLHF paper.
- Rafailov, R. et al. (2023). *Direct Preference Optimization: Your Language Model is Secretly a Reward Model.* NeurIPS.

Further Resources

› Chip Huyen: "RLHF: Reinforcement Learning from Human Feedback"; accessible technical overview

Prompting & Reasoning

Core Concepts

Prompting as the primary interface for working with LLMs. System prompts and their role in shaping behavior. Zero-shot vs. few-shot prompting; when examples matter. Structured outputs (JSON, XML) and why format matters. Temperature and sampling parameters. Prompt sensitivity and reproducibility. Chain-of-thought prompting: why "showing work" improves performance. Reasoning models (o1/o3, DeepSeek-R1, Claude extended thinking) and when to use them.

Core Readings

- Brown, T. et al. (2020). *Language Models are Few-Shot Learners.* NeurIPS. GPT-3; in-context learning.
- Wei, J. et al. (2022). *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models.* NeurIPS.

Further Resources

› Anthropic Prompt Engineering Guide; practical, regularly updated

› Anthropic: Extended thinking documentation; using Claude's reasoning mode

Evaluating & Choosing Models

Core Concepts

The benchmark landscape: MMLU (broad knowledge), HumanEval (code), GPQA (expert reasoning), Chatbot Arena (human preference). What benchmarks capture and miss: contamination, gaming, construct validity. Open vs. closed models: tradeoffs in capability, cost, transparency, privacy. Building task-specific evaluations for your research.

Core Readings

- Hendrycks, D. et al. (2021). *Measuring Massive Multitask Language Understanding.* ICLR. The MMLU benchmark.
- Chiang, W.-L. et al. (2024). *Chatbot Arena: An Open Platform for Evaluating LLMs through Human Preference.*

Further Resources

› Chatbot Arena Leaderboard; Artificial Analysis; live model rankings and cost comparisons

MODULE 3: DEPLOYING FOR RESEARCH

The practical infrastructure for adapting and running models, and your first research application.

Fine-Tuning & Deployment

Core Concepts

The decision framework: when to prompt-engineer vs. RAG vs. fine-tune. BERT-family models for classification—still the workhorse for many annotation tasks. Parameter-efficient fine-tuning (LoRA, QLoRA): dramatically reduced compute needs. SFT for custom instruction-following. Data requirements, compute costs, overfitting. The API transition: from interactive chat to programmatic access. Structured outputs, batching, and async patterns for large corpora. Cost estimation and rate limits. Self-hosted inference with vLLM/SGLang when privacy or scale demands it.

Core Readings

- Devlin, J. et al. (2019). *BERT: Pre-training of Deep Bidirectional Transformers.* NAACL. The encoder model for classification.
- Hu, E. et al. (2021). *LoRA: Low-Rank Adaptation of Large Language Models.* ICLR. Efficient fine-tuning.
- Kwon, W. et al. (2023). *Efficient Memory Management for Large Language Model Serving with PagedAttention.* SOSP. vLLM.

Further Resources

› Hugging Face PEFT library; LoRA, QLoRA, and other efficient methods

› Anthropic API documentation; OpenAI Cookbook; structured outputs, batching

Text Classification & Validation

Core Concepts

Zero-shot and few-shot classification with LLMs. Building labeling pipelines: prompt design, output parsing, error handling. The critical question: validation. Inter-annotator agreement metrics (Cohen's κ, Krippendorff's α) applied to LLM-human comparison. Systematic biases: positional bias, verbosity bias, cultural and political biases. Building gold-standard validation sets. Confidence calibration and uncertainty quantification.

Core Readings

- Gilardi, F., Alizadeh, M. & Haunss, S. (2023). *ChatGPT Outperforms Crowd-Workers for Text-Annotation Tasks.* PNAS.
- Pangakis, N., Wolken, S. & Fasching, N. (2023). *Automated Annotation with Generative AI Requires Validation.*

Further Resources

› Krippendorff, K. (2019) *Content Analysis*; foundational annotation methodology

› Scikit-learn classification metrics; Prodigy annotation tool

Social Science Application

- Ornstein, J., et al. (2023). *How to Train Your Stochastic Parrot: Large Language Models for Political Texts.* Building annotation pipelines at scale.

MODULE 4: SOCIAL SCIENCE APPLICATIONS

Applying LLMs to core research tasks, with rigorous attention to validation and methodology.

Information Extraction, Summarization & RAG

Core Concepts

Extracting structured information from unstructured text: entities, relationships, events, arguments. Summarization strategies and failure modes (hallucination, omission, distortion). Retrieval-Augmented Generation (RAG): using embeddings for semantic retrieval. Chunking strategies for effective retrieval. The RAG pipeline: embed → index → retrieve → generate. Working with large document corpora: legal texts, parliamentary records, news archives. Evaluation: measuring extraction and summarization quality.

Core Readings

- Lewis, P. et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.* NeurIPS. The foundational RAG paper.
- Gao, Y. et al. (2024). *Retrieval-Augmented Generation for Large Language Models: A Survey.* Comprehensive RAG overview.

Further Resources

› LangChain; LlamaIndex; RAG pipeline frameworks

› ChromaDB, FAISS, Pinecone; vector database options

› Anthropic: "Contextual Retrieval" (2024); improved chunking for RAG

MODULE 5: AGENTIC WORKFLOWS

Building autonomous research assistants and AI-augmented workflows.

Agents & Autonomous Workflows

Core Concepts

What agents are architecturally: tool use, planning loops, memory, and self-correction. The ReAct paradigm: reasoning + acting. Claude Code for code generation and project development. Agents for literature review: searching, summarizing, synthesizing. Hypothesis refinement and experimental design assistance. MCP (Model Context Protocol) and tool ecosystems. Environment setup: terminal basics, local vs. cloud (GitHub Codespaces as an accessible option).

Core Readings

- Yao, S. et al. (2023). *ReAct: Synergizing Reasoning and Acting in Language Models.* ICLR. The foundational agent framework.
- Schick, T. et al. (2023). *Toolformer: Language Models Can Learn to Use Tools.* NeurIPS.
- Anthropic (2024). *Model Context Protocol (MCP) Specification.* Open standard for AI-tool connections.

Further Resources

› Claude Code documentation; setup, usage, and best practices

› GitHub Codespaces; browser-based dev environments (no local setup)

› LangGraph; framework for building stateful agents

› Elicit, Semantic Scholar; AI-powered research tools

APPENDIX: RECOMMENDED TOOLS & PLATFORMS

Development & Notebooks

› Google Colab; primary platform for exercises, free GPU access

› GitHub Codespaces; cloud development environment for agent workflows

› VS Code + Python extensions; local development alternative

LLM APIs & Providers

› Anthropic API (Claude models)

› OpenAI API (GPT and o-series models)

› Google AI Studio (Gemini models)

› Together AI, Fireworks AI, Groq; open model inference providers

Key Python Libraries

› transformers (Hugging Face); model loading, tokenization, fine-tuning

› sentence-transformers; embedding models for semantic similarity and RAG

› scikit-learn; evaluation metrics, classification baselines

› peft; parameter-efficient fine-tuning (LoRA, QLoRA)

› langchain / llamaindex; RAG pipeline construction

Research & Literature Tools

› Semantic Scholar; AI-powered academic search

› Elicit; AI research assistant for literature review

› Connected Papers; visual exploration of citation networks